Lab Assignment 19.1

1. Python to Java Conversion

   Task:

   Translate a simple Python class into Java using AI assistance.

   Instructions:

   - Prompt AI to convert a Python class with attributes and methods into equivalent Java code.
   - Verify that constructors, method signatures, and data types are properly handled.
   - Run both versions (Python & Java) to compare expected outputs.

   Expected Output:

   - Equivalent working Java class translated from Python.

Screenshots:

assg_19.py > ...

```python
#write a python code to create a proper class and objects
class Student:
    def __init__(self, name, age, grade):
        self.name = name  # Student's name
        self.age = age    # Student's age
        self.grade = grade  # Student's grade

    def get_info(self):
        return f"Name: {self.name}, Age: {self.age}, Grade: {self.grade}"
# Create an instance of the Student class
student1 = Student("Alice", 20, "A")
# Print the student's information
print(student1.get_info())
```

```java

#convert the above into java code
public class Student {
    private String name; // Student's name
    private int age; // Student's age
    private String grade; // Student's grade

    // Constructor to initialize the student's attributes
    public Student(String name, int age, String grade) {
        this.name = name;
        this.age = age;
        this.grade = grade;
    }

    // Method to get the student's information
    public String getInfo() {
        return "Name: " + name + ", Age: " + age + ", Grade: " + grade;
    }

    // Main method to create an instance of Student and print info
    public static void main(String[] args) {
        Student student1 = new Student("Alice", 20, "A");
        System.out.println(student1.getInfo());
    }
}

```

Output:

```
Name: Alice, Age: 20, Grade: A
```

2. C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

- Ask AI to translate a recursive C++ factorial function into Python.
- Validate correctness by testing with sample inputs.
- Ensure Python version follows Pythonic conventions.

Expected Output:

- A correct Python implementation equivalent to the original C++ factorial function.

Screenshots:

```cpp
#write a c++ code function that calculates the factorial of a number
#include <iostream>
using namespace std;
// Function to calculate factorial
unsigned long long factorial(int n) {
    if (n < 0) {
        throw invalid_argument("Factorial is not defined for negative numbers.");
    }
    unsigned long long result = 1;
    for (int i = 1; i <= n; ++i) {
        result *= i;
    }
    return result;
}
int main() {
    int number;
    cout << "Enter a number to calculate its factorial: ";
    cin >> number;
    try {
        unsigned long long fact = factorial(number);
        cout << "Factorial of " << number << " is " << fact << endl;
    } catch (const invalid_argument& e) {
        cout << "Error: " << e.what() << endl;
    }
    return 0;
}
```

```python
#write above into python code
def factorial(n):
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers.")
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
try:
    number = int(input("Enter a number to calculate its factorial: "))
    fact = factorial(number)
    print(f"Factorial of {number} is {fact}")
except ValueError as e:
    print(f"Error: {e}")
```

Output:

```
Enter a number to calculate its factorial: 10
Factorial of 10 is 3628800
PS C:\Users\arell\Music\aiac>
```

3. SQL to Pandas Query

Task:

- Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

Screenshots:

```sql
#write a sql commands that create a table named as employee and insert some data and retrive the same and salary which are salary greater than 50000
-- Create the employee table
CREATE TABLE employee (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    position VARCHAR(50),
    salary DECIMAL(10, 2)
);
-- Insert some data into the employee table
INSERT INTO employee (id, name, position, salary) VALUES
(1, 'John Doe', 'Software Engineer', 60000.00),
(2, 'Jane Smith', 'Data Analyst', 45000.00),
(3, 'Emily Davis', 'Project Manager', 75000.00),
(4, 'Michael Brown', 'Intern', 30000.00);

-- Retrieve employees with salary greater than 50000
SELECT * FROM employee WHERE salary > 50000;
```

```python
#write the above sql commands into python using pandas dataframe
import pandas as pd
# Create a DataFrame to represent the employee table
data = {
    'id': [1, 2, 3, 4],
    'name': ['John Doe', 'Jane Smith', 'Emily Davis', 'Michael Brown'],
    'position': ['Software Engineer', 'Data Analyst', 'Project Manager', 'Intern'],
    'salary': [60000.00, 45000.00, 75000.00, 30000.00]
}
employee_df = pd.DataFrame(data)
# Retrieve employees with salary greater than 50000
high_salary_employees = employee_df[employee_df['salary'] > 50000]
print(high_salary_employees)
```

Output:

```
   id         name           position   salary
0   1     John Doe  Software Engineer  60000.0
2   3  Emily Davis    Project Manager  75000.0
PS C:\Users\arell\Music\aiac>
```

4. Real-Time Application: Multi-Language Snippet Converter

Scenario:

- Students will choose a real-time use case where they need code in multiple languages.

Example:

- Converting a Python web scraping snippet into JavaScript for browser automation.
- Translating a Java method into C# for enterprise applications.

Instructions:

- Prompt AI to translate code between two chosen languages.
- Test both implementations to ensure they produce the same results.
- Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

- Working equivalent code snippets in at least two different programming languages.

SCREENSHOTS:

```python
#write a code to calculate the fibonacci series up to n terms
def fibonacci(n):
    if n < 0:
        raise ValueError("Number of terms must be non-negative.")
    fib_series = []
    a, b = 0, 1
    for _ in range(n):
        fib_series.append(a)
        a, b = b, a + b
    return fib_series
try:
    n_terms = int(input("Enter the number of terms for Fibonacci series: "))
    print(f"Fibonacci series up to {n_terms} terms: {fibonacci(n_terms)}")
except ValueError as e:
    print(f"Error: {e}")
```

```java
#WRITE THE ABOVE CODE INTO JAVA
import java.util.Scanner;
public class Fibonacci {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the number of terms for Fibonacci series: ");
        int nTerms = scanner.nextInt();
        try {
            System.out.println("Fibonacci series up to " + nTerms + " terms: " + fibonacci(nTerms));
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
        scanner.close();
    }

    public static String fibonacci(int n) {
        if (n < 0) {
            throw new IllegalArgumentException("Number of terms must be non-negative.");
        }
        StringBuilder fibSeries = new StringBuilder();
        int a = 0, b = 1;
        for (int i = 0; i < n; i++) {
            fibSeries.append(a).append(" ");
            int temp = a;
            a = b;
            b = temp + b;
        }
        return fibSeries.toString().trim();
    }
}
```

Output:

```
PS C:\Users\arell\Music\aiac> cd "c:\Users\arell\Music\aiac\" ; if ($?) { javac Fibonacci.java } ; if ($?) { java Fibonacci }
Enter the number of terms for Fibonacci series: 10
Fibonacci series up to 10 terms: 0 1 1 2 3 5 8 13 21 34
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_19.py"
Enter the number of terms for Fibonacci series: 10
Fibonacci series up to 10 terms: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
PS C:\Users\arell\Music\aiac>
```